

Subroutines

Lecture – 10-11

Program to implement in Assembly

```
Int SomeFunc ( int a, int b, int c )
```

```
{
```

```
    int min = ... // code to find min of a,b and c here
```

```
    int max = ... // code to find max
```

```
    int total = ... // code to find total
```

```
    int avg = ... // code to find avg
```

```
    if ( avg > 0xC)
```

```
        return max
```

```
    else
```

```
        return min
```

```
}
```

```
Void main()
```

```
{
```

```
    int answer = SomeFunc(0xA,0xB, 0xC);
```

```
}
```

; subroutines call and ret instruction

[ORG 0x0100]

jmp start

a: db 0xA

b: db 0xB

c: db 0xC

min: db 0x0

max: db 0x0

total: db 0x0

avg: db 0x0

start: **call FindMin**

mov ax, 0x4C00 ;exit

int 0x21

FindMin: mov al, [a]

cmp al, [b]

ja blsSmaller ; b < a

cmp al, [c] ; either a or c is min

ja clsSmallest ; a > c

mov dl, al

jmp return

blsSmaller: mov al, [c]

cmp al, [b]

ja blsSmallest ; b < c

mov dl, [c] ; c is smallest

jmp return

blsSmallest: mov dl, [b]

jmp return

clsSmallest: mov dl, [c] ; dl = c

return: **mov [min], dl**

ret

Syntax to write function

FunctionLabel: <your code here>

<your code here>

<your code here>

ret

Start: **call FunctionLabel**

mov ax, 0x4C00 ;exit

int 0x21

Memory Configuration

Offset Registers: BX, SI, DI, BP, SP, IP

[BX], [SI], [DI] give data

[IP] is next instruction/opcode

Offset register contains address of data (or instruction)

Size of offset registers = 16-bit

Minimum address possible in offset register

Maximum address possible in offset register

X	X	X	X
---	---	---	---

0	0	0	0
---	---	---	---

F	F	F	F
---	---	---	---

Total memory space offset register can cover = $2^{16} = 2^6 \times 2^{10} = 64\text{KB}$

This memory space is called **Segment**.

Memory Configuration

Data of our program

Code of our program



0000	
0001	
0002	
...	
0009	
000A	
...	
000F	
0010	
0011	
...	
0100	1 st line of Our Program
0103	0xA
0104	0xB
0105	0xC
...	
0109	Avg
010A	start: call FindMin
...	Rest of our program here
0144 - 0145	INT 0x21 ; Exit
0146	Free Space

Memory Continued

Memory required by a program:

- 1- Code Segment
 - 2- Data Segment
 - 3- Stack Segment
- DOS defines the format of COM file of shared segment (i.e. one single segment contains code, data and stack)
 - Which offsets contain our code?
 - Which offsets contain our data?
 - Stack starts from offset 0xFFFF and it is decrementing stack

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	Free Space
FFF5	Free Space
FFF6	Free Space
FFF7	Free Space
FFF8	Free Space
FFF9	Free Space
FFFA	Free Space
FFFB	Free Space
FFFC	Free Space
FFFD	Free Space
FFFE	Free Space
FFFF	Free Space

Stack

- Stack operations are of word size.
- Push 16-bits on stack
- Pop 16-bits from stack
- 16-bit Data or address can be saved
- **Stack Pointer (SP)** contains the address of top of stack

(Stack Pointer) **SP** 

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	Free Space
FFF5	Free Space
FFF6	Free Space
FFF7	Free Space
FFF8	Free Space
FFF9	Free Space
FFFA	Free Space
FFFB	Free Space
FFFC	Free Space
FFFD	Free Space
FFFE	Free Space
FFFF	Free Space

Stack – Push Operation

Push **AX**:

1- $SP = SP - 2$

2- $[SP] = AX$

(Stack Pointer) **SP** 

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	Free Space
FFF5	Free Space
FFF6	Free Space
FFF7	Free Space
FFF8	Free Space
FFF9	Free Space
FFFA	Free Space
FFFB	Free Space
FFFC	New
FFFD	Data Here
FFFE	Free Space
FFFF	Free Space

Stack – Pop Operation

Pop **AX**:

1- $AX = [SP]$

2- $SP = SP + 2$

(Stack Pointer) **SP** 

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	Free Space
FFF5	Free Space
FFF6	Free Space
FFF7	Free Space
FFF8	Free Space
FFF9	Free Space
FFFA	Free Space
FFFB	Free Space
FFFC	Free Space
FFFD	Free Space
FFFE	Free Space
FFFF	Free Space

; subroutines call and ret instruction

[ORG 0x0100]

jmp start

a: db 0xA

b: db 0xB

c: db 0xC

min: db 0x0

max: db 0x0

total: db 0x0

avg: db 0x0

start: **call FindMin**

(0123) mov ax, 0x4C00 ;exit

int 0x21

(0456) FindMin: mov al, [a]

cmp al, [b]

ja blsSmaller ; b < a

cmp al, [c] ; either a or c is min

ja clsSmallest ; a > c

mov dl, al

jmp return

blsSmaller: mov al, [c]

cmp al, [b]

ja blsSmallest ; b < c

mov dl, [c] ; c is smallest

jmp return

blsSmallest: mov dl, [b]

jmp return

clsSmallest: mov dl, [c] ; dl = c

return: mov [min], dl

ret

Call Functionality

```
...  
(0456) FindMin:  mov al, [a]  
...  
start:    call FindMin  
(0123)  mov ax, 0x4C00 ;exit  
...
```

- ; Push Return add on stack**
- 1-** $SP = SP - 2$; Decrement SP
 - 2-** $[SP] = 0123$; Save return address
; Jump on FinMin label
 - 3-** $IP = 0456$;set next instruction address

(Stack Pointer) **SP** 

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	Free Space
FFF5	Free Space
FFF6	Free Space
FFF7	Free Space
FFF8	Free Space
FFF9	Free Space
FFFA	Free Space
FFFB	Free Space
FFFC	23
FFFD	01
FFFE	Free Space
FFFF	Free Space

Ret Functionality

```
...
return:  mov [min], dl
         ret
...
start:   call FindMin
(0123)  mov ax, 0x4C00 ;exit
         int 0x21
```

- ; Pop Return add into IP**
- 1-

IP = [SP] ; Save return address in IP
- 2-

SP = SP+2 ; Increment SP

; IP = 0123 means Jump on 0123

(Stack Pointer) **SP** 

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	Free Space
FFF5	Free Space
FFF6	Free Space
FFF7	Free Space
FFF8	Free Space
FFF9	Free Space
FFFA	Free Space
FFFB	Free Space
FFFC	Free Space
FFFD	Free Space
FFFE	Free Space
FFFF	Free Space

```

; multiple subroutines call and ret instruction - 4 function calls
[ORG 0x0100]

jmp start

a:    db 0x4
b:    db 0xA
c:    db 0xF
min:  db 0xF
max:  db 0x0
total: db 0x0
avg:  db 0x0
ans:  db 0x0

start:      call FindMin
(0193)     call FindMax
(0196)     call FindTotal
(0199)     call FindAvg
(019C)     cmp byte[avg], 0xC

            jae AvgIsGreater
            mov al, [min]
            jmp ReturnFunc

AvgIsGreater:      mov al, [max]

ReturnFunc:      mov [ans], al ; ans = max if avg > 0xc and min otherwise
                 MOV AX, 0x4C00 ; Terminate Program
                 INT 0x21

```

```

(010B)FindMin:  mov al, [a]

```

```

...

```

```

ret

```

```

(013F)FindMax:  mov al, [a]

```

```

...

```

```

ret

```

```

(0173)FindTotal:  mov al, 0

```

```

            add al, [a]

```

```

            add al, [b]

```

```

            add al, [c]

```

```

            mov [total], al

```

```

ret

```

```

(0185)FindAvg:    mov al, [total]

```

```

            mov cl, 3

```

```

            div cl      ;al = al/cl

```

```

            mov [avg], al

```

```

ret

```

```

jmp start
a:   db 0xA
b:   db 0xB
...
avg: db 0x0
ans: db 0x0

(0190)someFunc: call FindMin
(0193)          call FindMax
(0196)          call FindTotal
(0199)          call FindAvg
(019C)          cmp byte[avg], 0xC
                jae AvgIsGreater
                mov al, [min]
                jmp ReturnFunc

AvgIsGreater:   mov al, [max]
ReturnFunc:     mov [ans], al

                ret

start:          call someFunc
(01B3)          mov ax, 0x4C00

                INT 0x21

```

```

(010B)FindMin:  mov al, [a]
                ...
                ret
(013F)FindMax:  mov al, [a]
                ...
                ret

(0173)FindTotal: mov al, 0
                add al, [a]
                add al, [b]
                add al, [c]
                mov [total], al
                ret

(0185)FindAvg:  mov al, [total]
                mov cl, 3
                div cl      ;al = al/cl
                mov [avg], al
                ret

```

Program to implement in Assembly

```
Int SomeFunc ( int a, int b, int c )
```

```
{
```

```
    int min = ... // code to find min of a,b and c here
```

```
    int max = ... // code to find max
```

```
    int total = ... // code to find total
```

```
    int avg = ... // code to find avg
```

```
    if ( avg > 0xC)
```

```
        return max
```

```
    else
```

```
        return min
```

```
}
```

```
Void main()
```

```
{
```

```
//We have to pass these parameters
```

```
    int answer = Func(0xA,0xB, 0xC)
```

```
}
```

[ORG 0x0100]

jmp start

;1. a,b,c removed from global data

min: db 0xF

max: db 0x0

total: db 0x0

avg: db 0x0

ans: db 0x0

;---- 2. Passing (pushing) Parameters

start: *mov ax, 0xA*

push ax

mov ax, 0xB

push ax

mov ax, 0xC

push ax

call someFunc

mov ax, 0x4C00 int 0x21

someFunc: mov bp, sp???

;Read A,B,C parameters

;here ???

jmp FindMinAndMax

retFromFindMax: mov dx, 0

add dx, ax

...

mov [total], dx

;push ax???

mov al,

[total]

mov cl, 3

div cl *;al = al/cl*

mov [avg], al

;pop ax???

cmp byte[avg], 0xC

...

ReturnFunc: **mov [ans], al**

ret 6???

FindMinAndMax:

cmp ax, bx

... **mov [min], dx**

FindMax:

cmp ax, bx

...

mov [max], dx

jmp retFromFindMax

```
[ORG 0x0100]

jmp start

min:    db 0xF
max:    db 0x0
total:  db 0x0
avg: db 0x0
ans: db 0x0
;-----

start:  mov ax, 0xA
push ax

    mov ax, 0xB
    push ax

    mov ax, 0xC
    push ax

    call someFunc

    mov ax, 0x4C00    int 0x21
```

Start of Execution

SP →

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	Free Space
FFF5	Free Space
FFF6	Free Space
FFF7	Free Space
FFF8	Free Space
FFF9	Free Space
FFFA	Free Space
FFFB	Free Space
FFFC	Free Space
FFFD	Free Space
FFFE	Free Space
FFFF	Free Space

```
[ORG 0x0100]

jmp start

min:    db 0xF
max:    db 0x0
total:  db 0x0
avg: db 0x0
ans: db 0x0
;-----

start:  mov ax, 0xA
push ax

    mov ax, 0xB
    push ax
    mov ax, 0xC
    push ax
    call someFunc
    mov ax, 0x4C00    int 0x21
```

SP →

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	Free Space
FFF5	Free Space
FFF6	Free Space
FFF7	Free Space
FFF8	Free Space
FFF9	Free Space
FFFA	Free Space
FFFB	Free Space
FFFC	0A
FFFD	00
FFFE	Free Space
FFFF	Free Space

```
[ORG 0x0100]

jmp start

min:    db 0xF
max:    db 0x0
total:  db 0x0
avg: db 0x0
ans: db 0x0
;-----

start:  mov ax, 0xA
push ax

    mov ax, 0xB
    push ax

    mov ax, 0xC
    push ax

    call someFunc

    mov ax, 0x4C00    int 0x21
```

SP →

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	Free Space
FFF5	Free Space
FFF6	Free Space
FFF7	Free Space
FFF8	Free Space
FFF9	Free Space
FFFA	0B
FFFB	00
FFFC	0A
FFFD	00
FFFE	Free Space
FFFF	Free Space

```
[ORG 0x0100]

jmp start

min:    db 0xF
max:    db 0x0
total:  db 0x0
avg: db 0x0
ans: db 0x0
;-----

start:  mov ax, 0xA
push ax

    mov ax, 0xB
push ax

    mov ax, 0xC
push ax

    call someFunc

(0197)  mov ax, 0x4C00    int
0x21
```

SP →

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	Free Space
FFF5	Free Space
FFF6	Free Space
FFF7	Free Space
FFF8	0C
FFF9	00
FFFA	0B
FFFB	00
FFFC	0A
FFFD	00
FFFE	Free Space
FFFF	Free Space

```
[ORG 0x0100]

jmp start

min:    db 0xF
max:    db 0x0
total:  db 0x0
avg: db 0x0
ans: db 0x0
;-----

start:  mov ax, 0xA
push ax

    mov ax, 0xB
push ax

    mov ax, 0xC
push ax

    call someFunc

(0197)  mov ax, 0x4C00    int
0x21
```

SP →

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	Free Space
FFF5	Free Space
FFF6	97
FFF7	01
FFF8	0C
FFF9	00
FFFA	0B
FFFB	00
FFFC	0A
FFFD	00
FFFE	Free Space
FFFF	Free Space

```
someFunc:  mov bp, sp
           ;How can we read 3
;parameters and save them      ;in AX, BX
and CX???

           jmp FindMinAndMax
retFromFindMax: mov dx, 0
           add dx, ax
           ...
           mov [total], dx
           ;push ax???      mov al,
[total]
           mov cl, 3
           div cl      ;al = al/cl
           mov [avg], al
           ;pop ax???
           cmp byte[avg], 0xC
           ...
ReturnFunc: mov [ans], al
           ret 6???
```

Why mov bp, sp ???

SP →

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	Free Space
FFF5	Free Space
FFF6	97
FFF7	01
FFF8	0C
FFF9	00
FFFA	0B
FFFB	00
FFFC	0A
FFFD	00
FFFE	Free Space
FFFF	Free Space

```
someFunc:  mov bp, sp
           ;How can we read 3
;parameters and save them      ;in AX, BX
and CX???

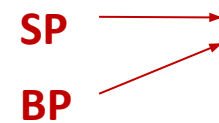
           jmp FindMinAndMax
retFromFindMax: mov dx, 0
           add dx, ax
           ...
           mov [total], dx
           ;push ax???           mov al,
[total]
           mov cl, 3
           div cl      ;al = al/cl
           mov [avg], al
           ;pop ax???
           cmp byte[avg], 0xC
           ...
ReturnFunc: mov [ans], al
           ret 6???
```

How can we use Base Pointer (BP) ???

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	Free Space
FFF5	Free Space
FFF6	97
FFF7	01
FFF8	0C
FFF9	00
FFFA	0B
FFFB	00
FFFC	0A
FFFD	00
FFFE	Free Space
FFFF	Free Space

```
someFunc:  mov bp, sp
           mov cx, [bp+2]
           mov bx, [bp+4]
           mov ax, [bp+6]

           jmp FindMinAndMax
retFromFindMax: mov dx, 0
               add dx, ax
               ...
               mov [total], dx
               ;push ax???
               mov al,
[total]
               mov cl, 3
               div cl    ;al = al/cl
               mov [avg], al
               ;pop ax???
               cmp byte[avg], 0xC
               ...
ReturnFunc:  mov [ans], al
           ret 6???
```



0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	Free Space
FFF5	Free Space
FFF6	97
FFF7	01
FFF8	0C
FFF9	00
FFFA	0B
FFFB	00
FFFC	0A
FFFD	00
FFFE	Free Space
FFFF	Free Space

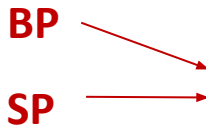
Base Pointer (BP) is Base of Activation Record of SomeFunc.
Why don't we use [sp+i] to read parameters???? Problem with Pop CX Pop BX Pop AX????

[ORG 0x0100] jmp start min: db 0xF max: db 0x0 total: db 0x0 avg: db 0x0 ans: db 0x0 ;----- start: mov ax, 0xA push ax mov ax, 0xB push ax mov ax, 0xC push ax call someFunc mov ax, 0x4C00 int 0x21	someFunc: mov bp, sp mov cx, [bp+2] bx, [bp+4] mov ax, [bp+6] jmp FindMinAndMax retFromFindMax: mov dx, 0 add dx, ax ... mov [total], dx;21 ;push ax??? mov cl, 3 div cl ;al = al/cl mov [avg], al ;pop ax??? cmp byte[avg], 0xC ... ReturnFunc: mov [ans], al ret 6??? [total] mov al,	FindMinAndMax: cmp ax, bx ... mov [min], dx FindMax: cmp ax, bx ... mov [max], dx jmp retFromFindMax Function has saved ans. How can we return now?
---	--	---

```
someFunc:  mov bp, sp
           mov cx, [bp+2]
           bx, [bp+4]
           mov ax, [bp+6]

           jmp FindMinAndMax
retFromFindMax: mov dx, 0
               add dx, ax
               ...
               mov [total], dx;21
               ;push ax???
               mov al,
[total]
               mov cl, 3
               div cl    ;al = al/cl
               mov [avg], al
               ;pop ax???
               cmp byte[avg], 0xC
               ...
ReturnFunc:  mov [ans], al
           ret 6 ;Vs ret ???
```

How will RET work
With this Configuration?



0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	Free Space
FFF5	Free Space
FFF6	97
FFF7	01
FFF8	0C
FFF9	00
FFFA	0B
FFFB	00
FFFC	0A
FFFD	00
FFFE	Free Space
FFFF	Free Space

Ret vs Ret 6

- Pop IP
- $SP = SP + 6$ in case of Ret 6
- This is Callee's responsibility to clear the parameters from stack.

Program to implement in Assembly

```
Int SomeFunc ( int a, int b, int c )
```

```
{
```

```
    int min = ... // code to find min of a,b and c here
```

```
    int max = ... // code to find max
```

```
    int total = ... // code to find total
```

```
    int avg = ... // code to find avg
```

```
    if ( avg > 0xC)
```

```
        return max
```

```
    else
```

```
        return min
```

```
}
```

```
Void main()
```

```
{
```

```
//SomeFunc has to return answer to main
```

```
    int answer = Func(0xA, 0xB, 0xC);
```

```
}
```

<pre> [ORG 0x0100] jmp start min: db 0xF max: db 0x0 total: db 0x0 avg: db 0x0 ; ans removed from global data ;----- start: ;Make temporary storage ;for return value on stack mov ax, 0xA push ax mov ax, 0xB push ax mov ax, 0xC push ax call someFunc ;Get return value here mov ax, 0x4C00 int 0x21 </pre>	<pre> someFunc: mov bp, sp mov cx, [bp+2] mov bx, [bp+4] mov ax, [bp+6] jmp FindMinAndMax retFromFindMax: mov dx, 0 add dx, ax ... mov [total], dx;21 ;push ax??? mov al, [total] mov cl, 3 div cl ;al = al/cl mov [avg], al ;pop ax??? cmp byte[avg], 0xC ... ReturnFunc: ;Set Return Value Here ret 6 </pre>	<pre> FindMinAndMax: cmp ax, bx ... mov [min], dx FindMax: cmp ax, bx ... mov [max], dx jmp retFromFindMax </pre>
--	---	---

[ORG 0x0100] jmp start min: db 0xF max: db 0x0 total: db 0x0 avg: db 0x0 ; ans removed from global data ;----- start: mov ax, 0xCCCC push ax mov ax, 0xA push ax mov ax, 0xB push ax mov ax, 0xC push ax call someFunc pop dx??? mov ax, 0x4C00 int 0x21	someFunc: mov bp, sp mov cx, [bp+2] bx, [bp+4] mov ax, [bp+6] jmp FindMinAndMax retFromFindMax: mov dx, 0 add dx, ax ... mov [total], dx;21 ;push ax??? [total] mov cl, 3 div cl ;al = al/cl mov [avg], al ;pop ax??? cmp byte[avg], 0xC ... ReturnFunc: mov ah,0 mov [bp+8], ax ret 6	FindMinAndMax: cmp ax, bx ... mov [min], dx FindMax: cmp ax, bx ... mov [max], dx jmp retFromFindMax
---	--	---

```
[ORG 0x0100]

jmp start

min:  db 0xF
max:  db 0x0
total: db 0x0
avg:  db 0x0

; ans removed from global data
;-----

start:  mov ax, 0xCCCC

        push ax

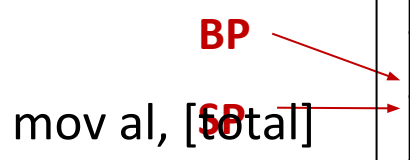
        mov ax, 0xA      push ax
        mov ax, 0xB
        push ax
        mov ax, 0xC
        push ax
        call someFunc
        pop dx
        mov ax, 0x4C00    int 0x21
```

BP
SP

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	97
FFF5	01
FFF6	0C
FFF7	00
FFF8	0B
FFF9	00
FFFA	0A
FFFB	00
FFFC	CC
FFFD	CC
FFFE	Free Space
FFFF	Free Space

```
someFunc:  mov bp, sp
           mov cx, [bp+2]
           mov bx, [bp+4]
           mov ax, [bp+6]

           jmp FindMinAndMax
retFromFindMax: mov dx, 0
              add dx, ax
              ...
              mov [total], dx;21
              ;push ax???
              mov cl, 3
              div cl    ;al = al/cl
              mov [avg], al
              ;pop ax???
              cmp byte[avg], 0xC
              ...
ReturnFunc: ;How can we set return value?
           ret 6
```



0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	97
FFF5	01
FFF6	0C
FFF7	00
FFF8	0B
FFF9	00
FFFA	0A
FFFB	00
FFFC	CC
FFFD	CC
FFFE	Free Space
FFFF	Free Space

Why not Ret 8????

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	97
FFF5	01
FFF6	0C
FFF7	00
FFF8	0B
FFF9	00
FFFA	0A
FFFB	00
FFFC	0A
FFFD	00
FFFE	Free Space
FFFF	Free Space

```
[ORG 0x0100]

jmp start

min:  db 0xF
max:  db 0x0
total: db 0x0
avg:  db 0x0

; ans removed from global data
;-----

start:  mov ax, 0xCCCC

        push ax

        mov ax, 0xA      push ax
        mov ax, 0xB
        push ax
        mov ax, 0xC
        push ax
        call someFunc
        pop dx
        mov ax, 0x4C00    int 0x21
```

Stack Configuration
After Return

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	97
FFF5	01
FFF6	0C
FFF7	00
FFF8	0B
FFF9	00
FFFA	0A
FFFB	00
FFFC	0A
FFFD	00
FFFE	Free Space
FFFF	Free Space

What if we miss
this Pop?

SP →

```
[ORG 0x0100]

jmp start

min:  db 0xF
max:  db 0x0
total: db 0x0
avg:  db 0x0

; ans removed from global data
;-----

start:  mov ax, 0xCCCC

        push ax

        mov ax, 0xA      push ax
        mov ax, 0xB
        push ax
        mov ax, 0xC
        push ax
        call someFunc
        pop dx
        mov ax, 0x4C00    int 0x21
```

After Pop

SP



0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	97
FFF5	01
FFF6	0C
FFF7	00
FFF8	0B
FFF9	00
FFFA	0A
FFFB	00
FFFC	0A
FFFD	00
FFFE	Free Space
FFFF	Free Space

Activation Record of SomeFunc

Reason???

Base of Activation Record
BP

Parameters located at BP + j

Return Value: BP + k

0147	Free Space
0148	Free Space
...	Free Space
FFEE	Free Space
FFEF	Free Space
FFF0	Free Space
FFF1	Free Space
FFF2	Free Space
FFF3	Free Space
FFF4	97
FFF5	01
FFF6	0C
FFF7	00
FFF8	0B
FFF9	00
FFFA	0A
FFFB	00
FFFC	0A
FFFD	00
FFFE	Free Space
FFFF	Free Space

Program to implement in Assembly

```
Int SomeFunc ( int a, int b, int c )
```

```
{
```

```
    //Now we have to save local variables
```

```
    int min = ... // code to find min of a,b and c here
```

```
    int max = ... // code to find max
```

```
    int total = ... // code to find total
```

```
    int avg = ... // code to find avg
```

```
    if ( avg > 0xC)
```

```
        return max
```

```
    else
```

```
        return min
```

```
}
```

```
Void main()
```

```
{
```

```
    int answer = Func(0xA,  
                      0xB, 0xC)
```

```
}
```

Local Variables

Local Variables located at BP - i

BP

Parameters located at BP + j

Return Value: BP + k

...	Free Space
FFEC	avg
FFED	
FFEE	total
FFEF	
FFF0	max
FFF1	
FFF2	min
FFF3	
FFF4	97
FFF5	01
FFF6	0C
FFF7	00
FFF8	0B
FFF9	00
FFFA	0A
FFFB	00
FFFC	CC
FFFD	CC
FFFE	Free Space
FFFF	Free Space

```
[ORG 0x0100]
jmp start

; min, max, total, avg removed
; from global data. Will be local
; variables now
;-----
start:  mov ax, 0xCCCC      push
ax
      mov ax, 0xA          push ax
      mov ax, 0xB
      push ax
      mov ax, 0xC
      push ax
      call someFunc
      pop dx
      mov ax, 0x4C00        int 0x21
```

```
someFunc:  mov bp, sp
           sub sp, 8????
           mov cx, [bp+2]
           mov bx, [bp+4]
           mov ax, [bp+6]
           jmp FindMinAndMax
retFromFindMax: mov dx, 0
           ...
           mov [total], dx;21
           ;push ax???
           mov al,
           mov cl, 3
           div cl    ;al = al/cl
           mov [avg], al
           ;pop ax???
           cmp byte[avg], 0xC
           ...
ReturnFunc: mov dh,0
           mov [bp+8], dx
           add sp, 8????
           ret 6
```

```
FindMinAndMax:
  cmp ax, bx
  ...      mov [min], dx

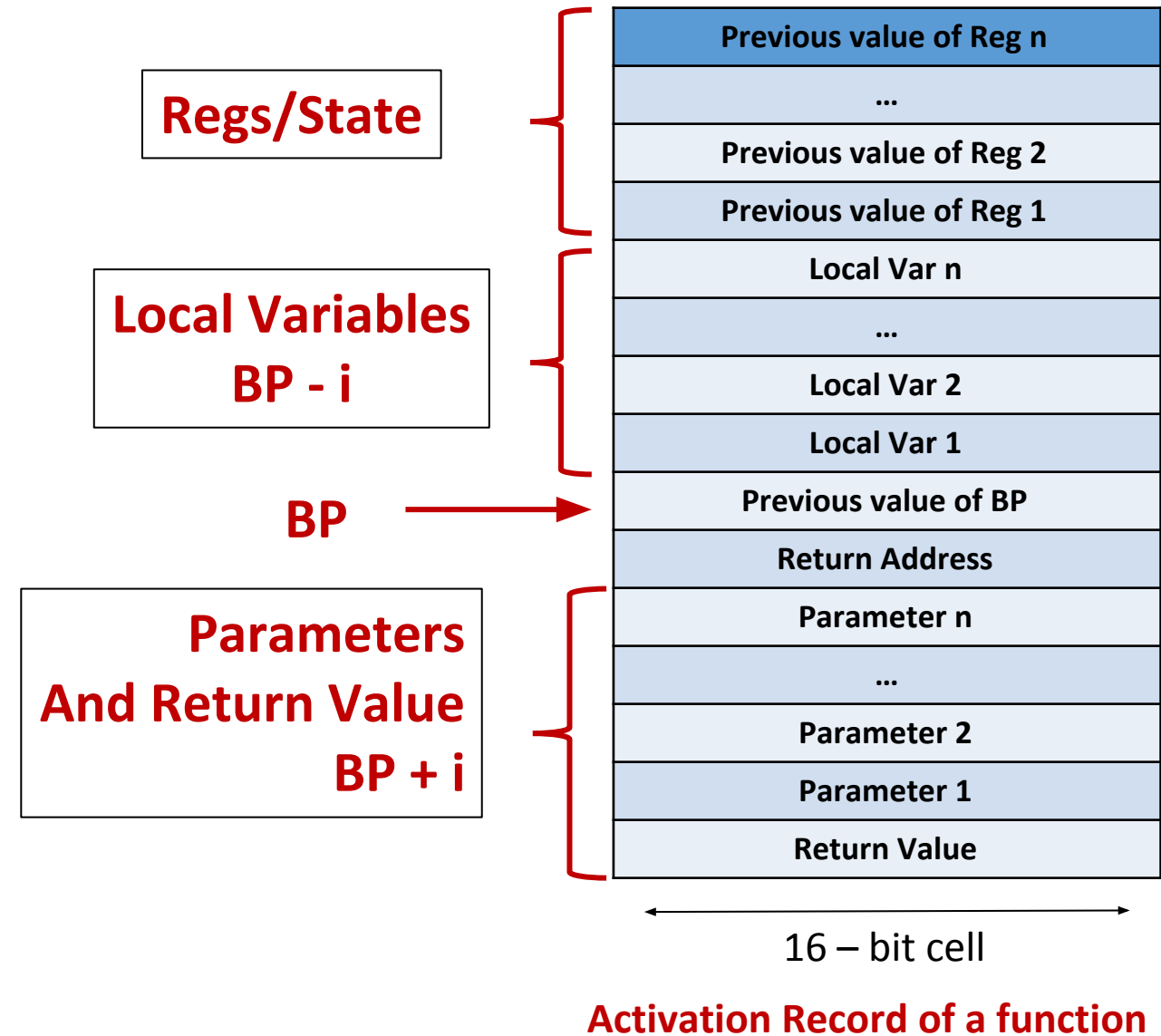
FindMax:
  cmp ax, bx
  ...
  mov [max], dx
  jmp retFromFindMax
```

How can we access these variables now?

[ORG 0x0100] jmp start ; min, max, total, avg removed from global data. Will be local variables now ;----- start: mov ax, 0xCCCC push ax mov ax, 0xA push ax mov ax, 0xB push ax mov ax, 0xC push ax call someFunc pop dx mov ax, 0x4C00 int 0x21	someFunc: mov bp, sp sub sp, 8 mov cx, [bp+2] mov bx, [bp+4] mov ax, [bp+6] jmp FindMinAndMax retFromFindMax: mov dx, 0 ... mov [bp-6], dx;21 ;push ax??? mov al, [bp-6] mov cl, 3 div cl ;al = al/cl mov [bp-8], al ;pop ax??? cmp byte[bp-8], 0xC ... ReturnFunc: mov dh,0 mov [bp+8], dx add sp, 8 ret 6	FindMinAndMax: cmp ax, bx ... mov [bp-2], dx FindMax: cmp ax, bx ... mov [bp-4], dx jmp retFromFindMax
--	--	---

Saving and Restoring Registers

- Callee has to save previous values of all registers it is going to use
- Push all these registers on stack
- Perform callee's functionality, freely use the registers
- Before leaving the function, pop all the values in their respective register to restore their original values



Subroutines – Caller's Responsibilities

1. Create Space for Return Value(s) (if required)
2. Push Parameters (if required)
3. Call Subroutine
4. Pop Return Values(s)

Subroutines – Callee's Responsibilities

1. Callee's Responsibility after entering the function
 - i. Push BP, Mark BP
 - ii. Create Space for Locals if required
 - iii. Save State
2. Perform Callee's Task
3. Callee's Responsibility before leaving the function
 - i. Restore State
 - ii. Release Space of Local Variables
 - iii. Pop/Restore BP
 - iv. Release Space of Parameters